

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT - ADVANCED

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

O QUE VEM POR AÍ?

Prepare-se para embarcar em uma transformação fascinante na maneira como desenvolvemos aplicações web. Nos próximos passos, você descobrirá uma técnica poderosa que promete revolucionar a performance e a otimização das suas criações. Imagine poder melhorar drasticamente o tempo de carregamento das páginas e proporcionar uma experiência de usuário incrivelmente fluida e eficiente, tudo isso enquanto aprimora o SEO da sua aplicação.

Vamos explorar juntos um universo onde a renderização não ocorre apenas no lado do cliente, mas é gerenciada pelo servidor. A jornada que temos pela frente vai te guiar desde os conceitos fundamentais até a implementação prática dessa técnica, passando por detalhes cruciais que farão toda a diferença no resultado do seu projeto.

A cada passo, você será introduzido(a) a ferramentas e configurações que tornarão seu ambiente de desenvolvimento mais robusto e preparado para os desafios modernos. Você verá como é possível migrar componentes essenciais do seu projeto atual para uma nova estrutura que maximiza eficiência e performance.

Ao longo do caminho, aprenderá a lidar com as nuances e complexidades que surgem, transformando obstáculos em oportunidades de aprendizado e crescimento. A prática constante será a chave, e cada pequeno ajuste e melhoria que faremos juntos fortalecerá seu entendimento e habilidade com essa nova abordagem.

HANDS ON

Prepare-se para mergulhar em uma transformação completa de um projeto React com Vite para a poderosa plataforma Next.js! Nos próximos três vídeos, você será guiado(a) passo a passo em uma jornada que vai além do básico, explorando o mundo do SSR (Server Side Rendering) e suas incríveis vantagens. Imagine melhorar a performance e o SEO das suas aplicações web ao mesmo tempo em que aprende técnicas avançadas e melhores práticas. É exatamente isso que você encontrará aqui.

No primeiro vídeo, começaremos com uma introdução ao SSR, explicando sua importância e como ele pode revolucionar a maneira como suas aplicações funcionam. Vamos revisar o projeto React que desenvolvemos na última aula e depois preparemos o ambiente de desenvolvimento necessário para essa migração.

Seguindo para o segundo vídeo, mergulharemos na implementação prática do SSR. Você aprenderá a configurar o Next.js para trabalhar com styled components e migrará o componente Banner do projeto React com Vite para o Next.js. Por meio de exemplos práticos, veremos como os componentes React são renderizados no servidor, melhorando a performance da sua aplicação.

No terceiro e último vídeo, exploraremos ainda mais conceitos de SSR e as melhores práticas para otimizar sua aplicação. Finalizaremos a migração do projeto, discutindo estratégias para melhorar ainda mais a performance e o SEO.

SAIBA MAIS

Next.js mais a fundo

Next.js é um poderoso framework de desenvolvimento que se destaca por sua capacidade de elevar o desenvolvimento de aplicações React a um novo nível. Construído sobre o popular React, Next.js traz uma série de funcionalidades que simplificam e otimizam o processo de desenvolvimento, tornando-o ideal para a criação de aplicações web modernas, rápidas e escaláveis.

No coração do Next.js está o React, uma biblioteca JavaScript para a construção de interfaces de usuário. Enquanto o React é focado na criação de componentes reutilizáveis e na renderização eficiente do lado do cliente, o Next.js expande essas capacidades ao introduzir a renderização do lado do servidor (SSR), a geração estática (SSG) e outras funcionalidades avançadas. Essa combinação permite que desenvolvedores criem aplicações que não apenas são rápidas e responsivas, mas também otimizadas para SEO e performance.

Principais conceitos

- **Renderização do lado do servidor (SSR):** SSR é uma técnica onde as páginas são renderizadas no servidor antes de serem enviadas ao cliente. Isso melhora significativamente o tempo de carregamento e a experiência do usuário, além de beneficiar o SEO da aplicação. Para entender melhor como o SSR é implementado no Next.js, [confira o artigo pilar sobre Next.js](#).
- **Geração estática (SSG):** Next.js permite a criação de páginas estáticas durante o build time. Isso significa que as páginas são geradas uma vez e servidas diretamente por um servidor de conteúdo estático, resultando em tempos de carregamento extremamente rápidos. Confira [esse artigo](#) no Medium explicando alguns conceitos importantes sobre o Next.js e suas siglas.
- **File-based routing:** no Next.js, a estrutura de diretórios define as rotas automaticamente. Isso simplifica a configuração e a [gestão de rotas](#), permitindo que desenvolvedores(as) se concentrem mais na lógica de negócios e menos na configuração.
- **API Routes:** Next.js facilita a criação de APIs internas, permitindo que pessoas desenvolvedoras construam backends robustos diretamente dentro da aplicação Next.js. Essa funcionalidade é especialmente útil para criar

aplicações fullstack, conforme detalhado no [artigo sobre aplicações fullstack com Next.js](#).

- Autenticação: a integração de OAuth 2.0 e OpenID Connect em aplicações Next.js é um processo simplificado, permitindo uma autenticação segura e eficiente. Para saber mais sobre como implementar essas autenticações, veja o [artigo sobre OAuth 2.0 e OpenID Connect](#).
- Performance e cache: Next.js é projetado para maximizar a performance, com técnicas de caching e otimização de assets. Para entender como essas práticas são aplicadas, consulte o [artigo sobre performance e cache](#).
- Segurança: A segurança é uma preocupação crítica em qualquer aplicação web. Next.js oferece várias ferramentas e práticas recomendadas para garantir que sua aplicação seja segura contra vulnerabilidades comuns. Para mais detalhes, veja o [artigo sobre segurança no Next.js](#).
- Depuração: depurar aplicações Next.js é um processo recorrente durante a vida de pessoas desenvolvedoras, e possuímos ferramentas integradas que facilitam a identificação e correção de problemas. O [artigo sobre como depurar apps Next.js](#) oferece um guia prático para essa tarefa.
- Server Actions: as server actions no Next.js permitem que ações específicas sejam executadas diretamente no servidor, melhorando a performance e a segurança da aplicação. Saiba mais sobre essa funcionalidade no [artigo sobre server actions](#).

Para quem deseja se aprofundar no desenvolvimento com Next.js, a Alura oferece uma série de cursos detalhados que cobrem desde os fundamentos até as técnicas mais avançadas. Você pode conferir a formação completa em Next.js [aqui](#).

Com Next.js, você não apenas expande as capacidades do React, mas também abre um leque de possibilidades para criar aplicações web rápidas, seguras e de alta performance. Está pronto(a) para levar suas habilidades de desenvolvimento para o próximo nível? Vamos começar essa jornada juntos!

Server Components no Next.js

Agora vamos explorar uma funcionalidade fascinante no Next.js: os Server Components. Eles nos permitem escrever interfaces de usuário que podem ser renderizadas e, opcionalmente, armazenadas em cache no servidor. No Next.js, o

trabalho de renderização é dividido por segmentos de rota para permitir o streaming e a renderização parcial. Existem três estratégias diferentes de renderização no servidor: estática, dinâmica e streaming. Vamos entender como os Server Components funcionam, quando usá-los e as diferentes estratégias de renderização.

Os Server Components trazem vários benefícios ao realizar a renderização no servidor. Primeiramente, eles permitem mover a busca de dados para o servidor, mais perto da fonte de dados. Isso pode melhorar o desempenho ao reduzir o tempo necessário para buscar os dados necessários para a renderização e o número de solicitações que o cliente precisa fazer. Além disso, manter dados e lógica sensíveis no servidor, como tokens e chaves de API, aumenta a segurança, evitando a exposição desses dados ao cliente. A renderização no servidor também permite armazenar o resultado em cache, reutilizando-o em requisições subsequentes e entre usuários, o que melhora o desempenho e reduz custos.

Outra vantagem significativa é a otimização do desempenho. Ao mover partes não interativas da interface de usuário para Server Components, reduzimos a quantidade de JavaScript necessário no cliente. Isso é benéfico para usuários com internet lenta ou dispositivos menos poderosos, já que o navegador terá menos JavaScript para baixar, analisar e executar. A renderização inicial da página e o First Contentful Paint (FCP) também são melhorados, pois podemos gerar HTML no servidor, permitindo que os usuários visualizem a página imediatamente, sem esperar que o JavaScript no cliente seja baixado, analisado e executado. Além disso, o HTML renderizado pode ser usado por bots de motores de busca para indexar suas páginas e por bots de redes sociais para gerar pré-visualizações de cartões sociais.

O streaming é outra funcionalidade interessante dos Server Components. Ele permite dividir o trabalho de renderização em partes e transmiti-las para o cliente à medida que ficam prontas. Isso permite que o usuário veja partes da página mais cedo, sem ter que esperar que toda a página seja renderizada no servidor.

No Next.js, os Server Components são usados por padrão, permitindo a implementação automática da renderização no servidor sem configuração adicional. Você pode optar por usar Client Components quando necessário. A renderização dos Server Components no Next.js utiliza as APIs do React para orquestrar a

renderização, dividindo o trabalho em partes por segmentos de rota individuais e Suspense Boundaries.

Cada parte é renderizada em duas etapas. Primeiro, o React renderiza os Server Components em um formato de dados especial chamado React Server Component Payload (RSC Payload). Em seguida, o Next.js usa o RSC Payload e as instruções de JavaScript dos Client Components para renderizar o HTML no servidor. No cliente, o HTML é usado para mostrar imediatamente uma pré-visualização rápida e não interativa da rota, apenas para a carga inicial da página. O RSC Payload é usado para reconciliar as árvores de Client e Server Components e atualizar o DOM, e as instruções de JavaScript são usadas para hidratar os Client Components e tornar a aplicação interativa.

O RSC Payload é uma representação binária compacta da árvore renderizada dos Server Components. Ele é usado pelo React no cliente para atualizar o DOM do navegador e contém o resultado renderizado dos Server Components, espaços reservados para onde os Client Components devem ser renderizados e referências aos seus arquivos JavaScript, além de quaisquer props passados de um Server Component para um Client Component.

As três estratégias de renderização no servidor são a renderização estática, dinâmica e streaming.

Na renderização estática, as rotas são renderizadas no momento da construção ou em segundo plano após a revalidação dos dados. O resultado é armazenado em cache e pode ser distribuído por uma CDN, sendo útil para rotas com dados que não são personalizados e podem ser conhecidos no momento da construção, como posts de blog ou páginas de produtos estáticas.

Já na renderização dinâmica, as rotas são renderizadas para cada usuário no momento da requisição, sendo útil para rotas com dados personalizados ou informações que só podem ser conhecidas no momento da requisição, como cookies ou parâmetros de busca da URL.

Na maioria dos sites, as rotas não são totalmente estáticas ou dinâmicas, mas um espectro. Por exemplo, uma página de e-commerce pode usar dados de produtos em cache revalidados periodicamente, mas também conter dados personalizados do cliente que não são armazenados em cache. No Next.js, você pode ter rotas

renderizadas dinamicamente que utilizam tanto dados em cache quanto não armazenados em cache. Isso ocorre porque o RSC Payload e os dados são armazenados em cache separadamente, permitindo optar pela renderização dinâmica sem se preocupar com o impacto de desempenho de buscar todos os dados no momento da requisição.

Durante a renderização, se uma função dinâmica ou uma solicitação de dados não armazenados em cache for detectada, o Next.js mudará para renderizar a rota dinamicamente. As funções dinâmicas dependem de informações conhecidas apenas no momento da requisição, como cookies do usuário, cabeçalhos da requisição atual ou parâmetros de busca da URL. No Next.js, usar `cookies()` e `headers()` em um Server Component fará com que toda a rota seja renderizada dinamicamente no momento da requisição.

O streaming, por sua vez, permite renderizar progressivamente a interface do usuário a partir do servidor. O trabalho é dividido em partes e transmitido ao cliente à medida que fica pronto, permitindo que o usuário veja partes da página imediatamente, antes que todo o conteúdo termine de ser renderizado. O streaming é integrado ao App Router do Next.js por padrão, ajudando a melhorar o desempenho da carga inicial da página, bem como a interface do usuário que depende de buscas de dados mais lentas.

Client Components no Next.js

Agora vamos falar sobre os Client Components no Next.js, uma funcionalidade que permite escrever interfaces de usuário interativas que são pré-renderizadas no servidor e podem usar JavaScript no cliente para rodar no navegador. Vamos entender como esses componentes funcionam, como são renderizados e quando você deve usá-los.

Os Client Components são extremamente úteis para criar interfaces interativas, pois podem usar estado, efeitos e listeners de eventos, oferecendo feedback imediato ao usuário e atualizando a UI conforme necessário. Além disso, eles têm acesso às APIs do navegador, como geolocalização ou `localStorage`, o que expande suas capacidades interativas.

Para usar Client Components no Next.js, você precisa adicionar a diretiva "use client" no topo do arquivo, acima dos seus imports. Essa diretiva declara uma fronteira entre módulos de Server e Client Components. Isso significa que, ao definir "use client" em um arquivo, todos os outros módulos importados para ele, incluindo componentes filhos, são considerados parte do bundle do cliente.

Vamos ver um exemplo prático para entender melhor. Imagine que temos um componente Counter que usa estado e um listener de evento:

```
'use client'

import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click
me</button>
    </div>
  )
}
```

Neste exemplo, usamos useState para gerenciar o estado do contador e um listener de evento onClick para atualizar o contador. A diretiva "use client" no topo do arquivo garante que esses recursos funcionem corretamente, já que, por padrão, todos os componentes no App Router são Server Components, onde essas APIs não estão disponíveis.

Quando você define a diretiva "use client" em um componente, todos os componentes filhos e módulos importados são considerados parte do bundle do cliente. Isso permite que você divida sua aplicação em múltiplos bundles de cliente conforme necessário.

A renderização dos Client Components no Next.js varia dependendo se a solicitação faz parte de um carregamento completo da página (uma visita inicial ao seu aplicativo ou um recarregamento de página acionado por uma atualização do navegador) ou uma navegação subsequente.

No carregamento completo da página, o Next.js usa as APIs do React para renderizar uma pré-visualização estática em HTML no servidor tanto para Client quanto para Server Components. Isso significa que, quando o usuário visita sua aplicação pela primeira vez, ele verá o conteúdo da página imediatamente, sem precisar esperar que o JavaScript dos Client Components seja baixado, analisado e executado.

No servidor, o React renderiza os Server Components em um formato de dados especial chamado React Server Component Payload (RSC Payload), que inclui referências aos Client Components. O Next.js usa o RSC Payload e as instruções de JavaScript dos Client Components para renderizar o HTML da rota no servidor. No cliente, o HTML é usado para mostrar imediatamente uma pré-visualização rápida e não interativa da rota. O RSC Payload é usado para reconciliar as árvores de Client e Server Components e atualizar o DOM, e as instruções de JavaScript são usadas para hidratar os Client Components e tornar a UI interativa.

A hidratação é o processo de anexar listeners de eventos ao DOM para tornar o HTML estático interativo. Nos bastidores, a hidratação é feita com a API `hydrateRoot` do React.

Em navegações subsequentes, os Client Components são renderizados inteiramente no cliente, sem o HTML renderizado no servidor. Isso significa que o bundle de JavaScript dos Client Components é baixado e analisado. Uma vez que o bundle esteja pronto, o React usa o RSC Payload para reconciliar as árvores de Client e Server Components e atualizar o DOM.

Às vezes, após declarar a fronteira "use client", pode ser necessário voltar ao ambiente do servidor. Por exemplo, você pode querer reduzir o tamanho do bundle do cliente, buscar dados no servidor ou usar uma API disponível apenas no servidor. É possível manter o código no servidor, mesmo que teoricamente ele esteja aninhado dentro de Client Components, intercalando Client e Server Components e Server Actions.

Os Client Components no Next.js oferecem uma maneira poderosa de criar interfaces interativas e responsivas, aproveitando ao máximo as capacidades do navegador. Usando a diretiva "use client" e entendendo como esses componentes são renderizados e hidratados, você pode melhorar significativamente a experiência do usuário em suas aplicações.

Server Actions e Mutations no Next.js

Vamos explorar outro recurso poderoso do Next.js: Server Actions e Mutations. Essas funções assíncronas são executadas no servidor e podem ser usadas tanto em Server quanto em Client Components para lidar com submissões de formulários e mutações de dados em aplicações Next.js.

Definição e convenção

Uma Server Action pode ser definida com a diretiva "use server" do React. Você pode colocar essa diretiva no topo de uma função assíncrona para marcá-la como uma Server Action ou no topo de um arquivo separado para marcar todas as exportações desse arquivo como Server Actions. Por exemplo, em um Server Component, você pode usar a diretiva "use server" no nível da função:

```
// Server Component
export default function Page() {
  // Server Action
  async function create() {
    'use server'
    // ...
  }
  return (
    // ...
  )
}
```

Em Client Components, você só pode importar ações que utilizam a diretiva "use server" no nível do módulo. Isso significa criar um arquivo e adicionar a diretiva "use

server" no topo dele. Todas as funções dentro desse arquivo serão marcadas como Server Actions que podem ser reutilizadas em Client e Server Components:

```
// app/actions.ts
'use server'

export async function create() {
  // ...
}
```

Comportamento

As Server Actions podem ser chamadas utilizando o atributo action em um elemento <form>. Nos Server Components, isso oferece suporte ao aprimoramento progressivo por padrão, o que significa que o formulário será enviado mesmo se o JavaScript ainda não tiver sido carregado ou estiver desativado. Nos Client Components, os formulários que invocam Server Actions irão enfileirar submissões se o JavaScript não estiver carregado ainda, priorizando a hidratação do cliente. Após a hidratação, o navegador não será atualizado na submissão do formulário.

As Server Actions não se limitam a <form> e podem ser invocadas de manipuladores de eventos, useEffect, bibliotecas de terceiros e outros elementos de formulário, como <button>. Elas integram-se com a arquitetura de cache e revalidação do Next.js. Quando uma ação é invocada, o Next.js pode retornar tanto a UI atualizada quanto novos dados em uma única rodada de servidor. Por trás dos panos, as ações utilizam o método POST e apenas este método HTTP pode invocá-las. Os argumentos e o valor de retorno das Server Actions devem ser serializáveis pelo React.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, embarcamos em uma transformação completa de um projeto React com Vite para a poderosa plataforma Next.js, com foco especial na implementação de Server Side Rendering (SSR). Ao longo do conteúdo, aprofundamos nosso entendimento sobre SSR e suas vantagens significativas, incluindo melhorias substanciais na performance e otimização de SEO das aplicações web.

Começamos com uma introdução ao conceito de SSR, explicando sua importância e como ele pode revolucionar a maneira como suas aplicações funcionam. Revisamos o projeto React que desenvolvemos anteriormente, preparando o ambiente de desenvolvimento necessário para a migração para Next.js. Esta preparação foi essencial para garantir que todos os componentes e dependências estivessem configurados corretamente para suportar SSR, estabelecendo uma base sólida para a migração.

Com o ambiente configurado, mergulhamos na implementação prática do SSR. Aprendemos a configurar o Next.js para trabalhar com styled components e migramos o componente Banner do projeto React com Vite para o Next.js. Por meio de exemplos práticos, vimos como os componentes React são renderizados no servidor, melhorando significativamente a performance da aplicação. Essa prática demonstrou claramente a diferença que o SSR pode fazer na experiência do usuário, proporcionando uma renderização mais rápida e eficiente das páginas.

Além da implementação, exploramos conceitos avançados e melhores práticas para otimizar ainda mais nossa aplicação. Discutimos estratégias para melhorar a performance e o SEO, garantindo que a aplicação não apenas funcione bem, mas também seja rápida e otimizada para motores de busca. Essas técnicas são cruciais para aumentar a visibilidade e a usabilidade da aplicação, tornando-a mais atrativa para os usuários finais.

Ao final desta aula, você adquiriu uma compreensão sólida de como migrar um projeto React para Next.js utilizando SSR. Aprendeu a configurar o ambiente, implementar e testar a renderização do lado do servidor, e aplicou melhores práticas para otimização de performance e SEO. Com essas habilidades, você está preparado para desenvolver aplicações web mais rápidas, eficientes e robustas, elevando suas competências de desenvolvimento ao próximo nível.

REFERÊNCIAS

NEXT.JS. **Authentication.** 2023. Disponível em:
<https://nextjs.org/docs/authentication>. Acesso em: 26 jun. 2024.

NEXT.JS. **Server-side Rendering (SSR).** 2023. Disponível em:
<https://nextjs.org/docs/basic-features/pages#server-side-rendering>. Acesso em: 26 jun. 2024.

NEXT.JS. **Documentation. Static Site Generation (SSG).** 2023. Disponível em:
<https://nextjs.org/docs/basic-features/pages#static-generation>. Acesso em: 26 jun. 2024.

PALAVRAS-CHAVE

Next.js, React, Server-Side Rendering (SSR).

EMEND

POSTECH